

Audit research notes

Matteo Franzil, Valentino Armani, Luis Augusto Dias Knob, Domenico Siracusa

June 19, 2024

Contents

1	TODOs	1
1.1	DONE SoA research on log clustering, graph representation, and mapping	1
1.2	SCHE Strengthen the motivation for the project	1
1.3	DONE Implement a rough parser and filter out background noise	1
1.4	DONE Investigate difference between <code>kubect1</code> and other tools	2
1.5	SCHE Start collecting data of class 1	2
1.6	SCHE Start implementing a ML model for learning templates	2
1.7	Other things	2
2	Strengthening the motivation for the project	2
3	Behaviour analysis of the audit logs	3
3.1	Generic	3
3.2	Namespaces	3
3.3	Other quirks	3
3.4	Impersonation	4
3.5	Control plane components	4
4	Future work	4

1 TODOs

1.1 **DONE** SoA research on log clustering, graph representation, and mapping

1.1.1 **DONE** Inspect SoA for what other people did in the field

- We did some SoA research to understand if other works performed log clustering and pattern matching. The results are quite unsatisfactory: most works - obviously - focus on NLP processing for the log content, few attempt to actually clusterize multiple lines.

[1] does so in a Hadoop context, a distributed MapReduce system where failures are common. Starting from the assumption that events are limited in quantity, they vectorize sequences over N dimensions (N different events) and clusterize them using similarity.

[2] LogMiner (first author also wrote [1]) also does something similar to what we want to do: transform log sequences into *behaviours*, depending on the context the various items acted upon. They again use inverse document frequency to assign weight to logs in each cluster, and partitions behaviours using PIDs. The amount of behaviours is, again, quite limited.

1. Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, “Log clustering based problem identification for online service systems,” in Proceedings of the 38th International Conference on Software Engineering Companion, in ICSE ’16. New York, NY, USA: Association for

Computing Machinery, May 2016, pp. 102–111. doi: 10.1145/2889160.2889232. Available: <https://dl.acm.org/doi/10.1145/2889160.2889232>. [Accessed: Jun. 18, 2024]

2. H. Zhang, L. Cai, L. Zhao, A. Yu, J. Ma, and D. Meng, “LogMiner: A System Audit Log Reduction Strategy Based on Behavior Pattern Mining,” in MILCOM 2022 - 2022 IEEE Military Communications Conference (MILCOM), Rockville, MD, USA: IEEE, Nov. 2022, pp. 292–297. doi: 10.1109/MILCOM55135.2022.10017626. Available: <https://ieeexplore.ieee.org/document/10017626/>. [Accessed: May 09, 2024]

1.2 SCHE Strengthen the motivation for the project

See motivation for a detailed explanation of the motivation behind the project. This section is WIP.

1.3 DONE Implement a rough parser and filter out background noise

1.3.1 DONE Finish clustering non-relevant events and implement them in the parser

We now have a parser that filters out background noise and separates control plane traffic from user traffic. The control plane traffic will be unlabeled, while the user traffic will be labeled depending on the behavior.

1.4 DONE Investigate difference between kubectl and other tools

- `kubectl` performs a series of sanity checks before actually sending the request to the API server. This is done by the client itself, for example, getting the namespace and ResourceQuotas before creating an object.
- On the other hand, other behaviours are managed by the KCM, for example, creating a secret for a ServiceAccount, or creating a ConfigMap for the root CA.

1.4.1 DONE Try to use curl or an external library

- Does not really change much, user agent apart. Some behaviours are still triggered by the KCM, and API calls made have no effect on his decisions.

1.4.2 FAIL Inspect code to see how api calls are made

- Utter chaos, depending on the command the API calls are different: sometimes it uses the API objects, sometimes raw HTTP requests... plus it uses command builders that are not easy to follow.

1.5 SCHE Start collecting data of class 1

1.5.1 DONE Take notes of weird behaviours

Have a look at behaviour analysis for a detailed list of weird behaviours that we have found in the logs.

1.5.2 TODO Label the data

1.6 SCHE Start implementing a ML model for learning templates

1.6.1 TODO Try with CNN

1.6.2 FAIL Scrapped: learn templates from the logs

- Idea: use a ML model to generalize and attempt to match 'templates' of behaviour in the logs. For example: we already have an idea of how namespaces are created, deleted, and so on. We

could use a model to learn these templates and then match them in the logs. If no match is found, then an attempt could be made to generate a new template based on the most similar ones.

1.7 Other things

1.7.1 DELE Ask PoliTO for a decent dataset

- Stefano has contacted their IT guys, but it looks like they have no remote idea what an audit log is. I explained it to them, let's see...

1.7.2 TODO Start filtering out users also by group

2 Strengthening the motivation for the project

- Signature-based detection is not enough. To demonstrate so, we can use the example of Falco. Glossing over the nightmare that is installing it, Falco comes with a pre-set of rules that are fairly basic. For example, rules are triggered when SAs are created and deleted, secrets are accessed, and so on. However, these rules are exceedingly verbose and alerts are triggered for use cases that are not malicious at all. For example, Flannel renewing SA tokens, or the kube-controller-manager creating a secret for a SA when a namespace is created, are all examples of false positives that are triggered by Falco.
- ValidatingWebhookConfiguration is a powerful tool that can be used to validate requests to the API server. However, it is not used in the default installation of Kubernetes, and it is not easy to set up. Furthermore, it is not easy to understand what is happening when a request is denied, as the logs are not very verbose.

3 Behaviour analysis of the audit logs

3.1 Generic

- **watch** requests are long-running and are usually made by control plane components. Since the **ResponseStarted** and **ResponseComplete** events can be in different times, we chose to **remove** the **ResponseStarted** event from the logs.
 - Indeed, for us it is important to have the **requestReceivedTimestamp** that is present in all events anyway. With that, we can understand when the request was made and when it was completed.
 - **ResponseStarted** appears in **watch** requests, in **kubectl exec** requests, and in **delete** requests. In the latter case, **ResponseComplete** is sent once the object is actually deleted.
- **kubectl** requests are internally translated into a set of API calls that are made by the **kube-apiserver**. Further subsections detail such calls and how they can help us derive templates for the parser.

3.2 Namespaces

- Every time a namespace is created, the following things happen:
 - A **list** request is made to **/api/v1/namespaces/{}/resourcequotas** by the **kube-apiserver** for no apparent reason
 - * To add insult to injury, usually this line is out of order in the log
 - The **kube-controller-manager** gets the **service-account-controller** SA and generates a token for it

- The SA `root-ca-publisher` puts the `root-ca.crt` in a ConfigMap in the namespace
- The `service-account-controller` creates a SA called `default` in the namespace
- When a namespace is deleted, all subresources are deleted with a `deletecollection` request followed by a `list`. Exceptions include `Pods`, that weirdly are deleted by a `list`, then `deletecollection`, then `list` again. Also, when `ConfigMaps` are deleted, the `kube-controller-manager` (and not the `root-ca-cert-publisher`) tries to create again the CM that was just deleted, but is rejected because the namespace is being terminated.

3.3 Other quirks

- When a secret is created for a SA the secret is instantly patched by the `kube-controller-manager`, which is in charge of actually generating that secret along with an object like

```
{"data":{"ca.crt":"...", "namespace":"...", "token":"..."}
```

where the three fields are base64 encoded.

- Secrets can be traced back to the Pod they have been mounted into by either looking at the spec or monitoring for `watch` requests by the node that is hosting the pod.
- Users with access to `kubectl exec` can access the secrets of the pods they are exec-ing into using `printenv`.
- `ResourceQuotas` can be used for more than just limiting resources, they can also be used to limit the number of objects that can be created. This also applies to things like `ServiceAccounts`: thus, it seems that every time a potentially limitable object is created, a `get` request is made to `/api/v1/{object}` to assess the current state.
- When CSRs are being approved or denied, the `update` verb is used and the actual decision is in the payload of the request. This makes it less straightforward to understand if the request was allowed or denied.
- When old-style `ServiceAccount` tokens are used, instead of the new CSR style `Users`, the API server detects the *offending* action and adds a label `kubernetes.io/legacy-token-last-used` to the Secret.

3.4 Impersonation

- Users with access to the `impersonate` verb and one of `users`, `groups`, and `serviceaccounts` can impersonate other identities.
- This is correctly reflected in the audit logs, but could be a problem implementation-wise.

3.5 Control plane components

- Control plane components' "background noise" has been filtered out and the details are available in [queries.org](https://kubernetes.io/docs/reference/audit-logs/).

4 Future work

- Using KWOK
- Ask for a master's student to create a simulator with KWOK, defining scenarios and gathering the audit log